The ML System Design Rubric

Scaler School of Technology | ML System Design and LLMOps | Week 8 — Framework Reference

This rubric is a **structured 10-step framework** for answering any ML system design question. Follow the steps in order. Interviewers evaluate your answer on **structure, completeness, and ability to articulate trade-offs** — not on proposing the most complex model.

1 Clarify the Problem

Ask questions before designing. Define scope, constraints, and success criteria. What must the system achieve? What metrics define success? What is the escalation path when the system fails?

2 Context

Provide domain and business context. Why does this system exist? What is the business objective? Who are the users? What is the cost of a wrong action?

3 Problem Statement

Formally define the ML problem. What is the input, what is the output, and what decision does the agent need to make at each step?

4 Define Requirements

Split into two categories. **Functional:** what the system must do (intent classification, tool use, escalation). **Non-functional:** latency SLA, scale, cost ceiling, auditability, availability.

5 Estimations

Back-of-envelope sizing. Concurrent sessions, tokens/day, LLM cost/hour, tool call volume. These numbers expose whether the design is financially feasible.

6 System Architecture

Design the end-to-end system. Decompose into: **(a) Data Collection** — conversation history, tool results; **(b) Data Engineering** — state management, context window; **(c) Feature Engineering** — customer context, KB; **(d) Preprocessing** — intent routing; **(e) Models** — classifier + LLM orchestrator + tools; **(f) Post-processing** — safety, audit, escalation.

7 Evaluation: Offline and Online

Offline: intent accuracy, tool selection accuracy, resolution rate on golden test set. Online: resolution rate, CSAT score, cost per conversation, escalation rate.

8 A/B Testing

Design the experiment. Bias to flag: severity bias (AI resolves easy cases, humans get hard ones — inflates AI metrics). Track both.

9 Deployment and Monitoring

Stateless pods + Redis session state. Monitor: turns-per-conversation, cost/conversation, tool API error rates, escalation rate by intent, CSAT rolling average.

10 Risk Management and Trade-offs

Autonomous vs. human-approval for high-stakes actions. Loop detection (max turns + repeated action detection + cost ceiling). Prompt injection defences. When NOT to use agents.

How to use this rubric: Work through Steps 1–3 aloud. Spend most time on Steps 4–6. Steps 7–10 differentiate senior candidates. For agentic systems, Step 10 is especially important — agents fail in production in ways models don't.

Case Study: Multi-model Agentic Pipeline (AI Customer Support)

Scaler School of Technology | ML System Design and LLMOps | Week 8 — Reference Solution

Step 1 | Clarify the Problem

Given problem: Design a production AI customer support agent for an e-commerce platform with 2 million customers.

Clarifying questions:

- Can the agent take autonomous actions (issue refunds), or only recommend? (*Autonomous for low-risk; human approval for refunds > $50*)
- What is the escalation SLA to a human agent? (*Within 60 seconds of trigger*)
- What languages must be supported? (*English + top 5 by user volume*)
- What defines a "resolved" conversation? (*Customer did not re-contact within 24h*)
- Is there an audit requirement? (*Yes — every action logged for compliance*)

Step 2 & 3 | Context and Problem Statement

Context: Customer support is a cost centre being partially automated. Wrong actions (unwarranted refunds, sharing wrong order details, not escalating an angry customer) cause direct financial loss and trust damage. The agent must be *conservative* — better to escalate than to act incorrectly.

Problem Statement: Given a customer message and conversation history, classify intent, retrieve relevant context (order data, product info, policies), and either resolve the issue via API action or generate a helpful response. If resolution fails or confidence is low, escalate to a human agent. Input: message + session context + customer ID. Output: response text + optional action taken + escalation flag + audit log entry.

Step 4 | Define Requirements

Requirement	Implication for design
Functional	
Intent classification	Order status / returns / product questions / complaints / other
Tool use	Order lookup, refund API, product KB search, escalation trigger
Multi-turn conversation	Session state must persist across turns (Redis)
Audit log of every action	Every tool call + result + LLM response logged immutably
Language detection + translation	Pre-processing step before intent classification
Non-functional	
p99 < 5s per turn	LLM + tool calls must fit within budget
10,000 concurrent conversations	Stateless pods; state in Redis
Cost ceiling $0.05/resolved conversation	Per-conversation cost monitor with hard cutoff
Human escalation within 60s	Escalation queue with SLA monitoring

Step 5 | Estimations

- **Sessions:** 10K concurrent × avg 5 turns = **50K turns/session-set**
- **LLM tokens:** 10K concurrent × 1K tokens/turn × 5 turns = **50M tokens/hour**
- **LLM cost:** at $0.15/1M tokens = **$7.50/hour** for LLM alone
- **Tool calls:** avg 2 tool calls/conversation × 3,333 new conv/min = **6,666 tool API calls/min**
- **Session state:** 10K sessions × 10KB context = **100MB in Redis** (easily fits)
- **Cost per conversation:** $7.50/hr ÷ (3,333 conv/min × 60 min) ≈ **$0.037/conv** — within $0.05 ceiling

Step 6 | System Architecture

The key architectural insight: Agents fail in production in ways models don't. The four failure modes are: **(1) unbounded loops** — agent calls tools forever; **(2) cost blow-up** — one bad query runs up $50 in LLM calls; **(3) context overflow** — conversation grows until it hits the context window; **(4) tool failure cascade** — order API timeout stalls the entire agent. **Design for bounded execution, not optimal execution.**

6a Data Collection & State Management

Sources: Customer messages (chat API), order data (OMS), product data (catalogue API), policies (internal KB), conversation history (Redis, TTL 24h).

Context window management: Sliding window keeps last N turns. Older turns summarised into a compressed memory string. Hard cap: if session exceeds 15 turns, force escalation.

Customer context: Order history, account tier, past issues fetched at session start and injected into system prompt. Does not change during session.

6b & 6c Data Engineering & Feature Engineering

Tool result caching: Order status cached 30s (same customer may ask again). Product info cached 5 min. Policy answers cached 1h.

Product KB: Product descriptions + FAQs embedded (text-embedding-3-small) and indexed in FAISS. Queried for product-related intents.

PII scrubbing: All conversation logs scrubbed of PII (credit card numbers, addresses) before writing to audit store.

6d & 6e Preprocessing / Intent Routing & Models

Intent classifier: Fine-tuned DistilBERT ($<$ 20ms). 5 intents: order_status, returns, product_question, complaint, other. Routes: order_status $\rightarrow$ direct tool call (no LLM needed); product_question $\rightarrow$ KB retrieval + LLM; complaint $\rightarrow$ full agent loop; other $\rightarrow$ immediate escalation.

Main LLM: GPT-4o-mini with function calling. Tools: `get_order_status(order_id)`, `initiate_return(order_id, reason)`, `issue_refund(order_id, amount)`, `search_product_kb(query)`, `escalate_to_human(reason)`.

Translation: Helsinki-NLP/opus-mt for non-English input (pre-processing) and output (post-processing). Adds $\sim$100ms.

Loop detection: If agent calls the same tool with the same arguments twice in one session, break loop and escalate.

6f Post-processing & Guardrails

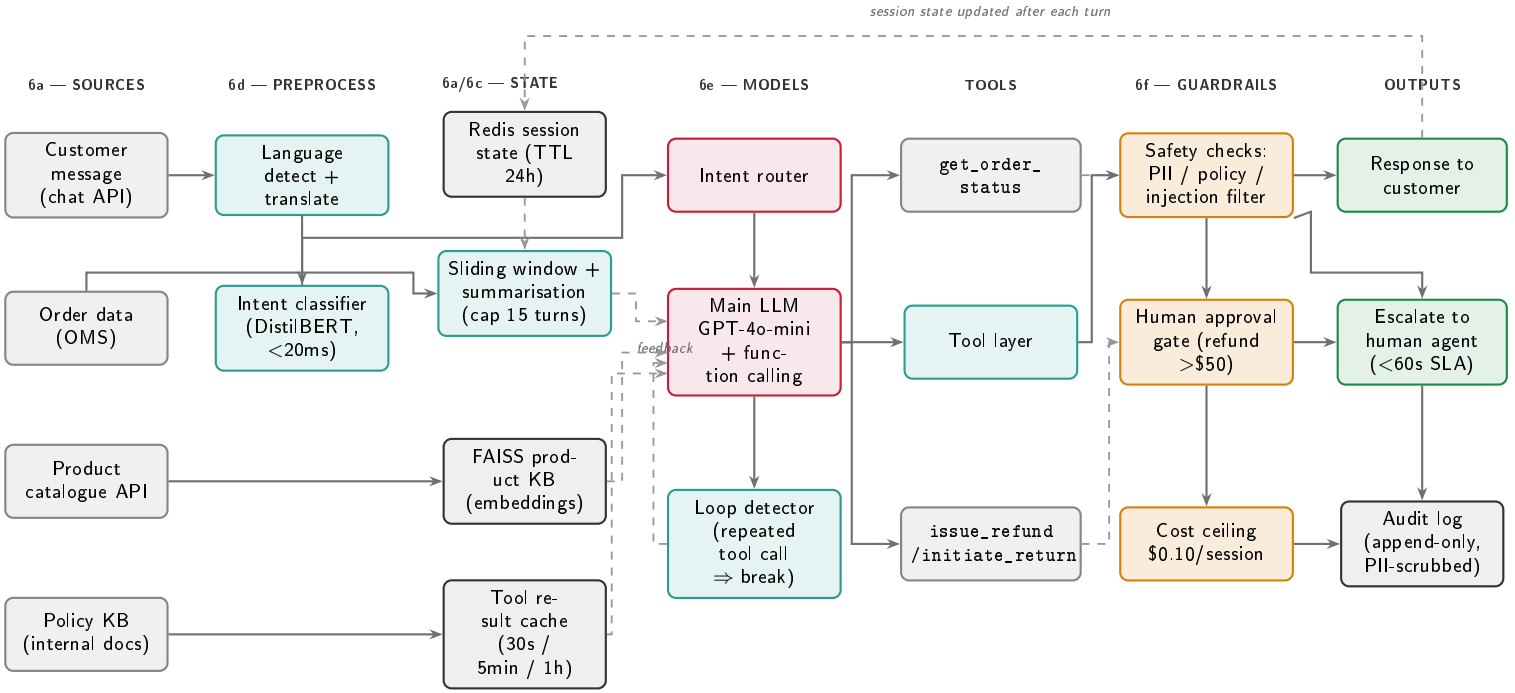
Action confirmation: Refunds $>$ $50 require human approval before execution. Agent informs customer: "I can initiate this refund but it requires manager approval — I've triggered that process."

Safety checks: PII in response (block), policy violation (block + escalate), prompt injection detection (`ignore previous instructions` patterns).

Audit log: Every tool call + arguments + result + LLM response + latency written immutably to append-only store.

Cost ceiling: If accumulated LLM token cost for session exceeds $0.10, force graceful escalation.

Step 6 | System Architecture — End-to-End Diagram



Reading the diagram: Solid arrows = primary request flow (message → response). Dashed arrows = read-only context lookups and feedback (state reads, KB retrieval, session updates, loop-detector feedback into the LLM). Every guardrail node (6f, amber) has two exits: forward to the customer *or* sideways to escalation — this is what "bounded execution" looks like structurally.

Step 7 | Evaluation: Offline and Online

Type	Metric	Why it matters
Offline	Intent accuracy	Does the classifier route correctly?
	Tool selection accuracy	Does the agent call the right tool?
	Resolution rate on golden set	End-to-end test on labelled conversation set
Online	**Resolution rate (north star)**	% conversations resolved without human
	CSAT score	Post-conversation rating (1–5 stars)
	Cost per resolved conversation	Must stay $< \$0.05$ ceiling

Guardrails: Escalation rate (if $< 5\%$ = agent is overconfident; if $> 40\%$ = agent is too cautious). Average turns per conversation (if > 8 = agent is stuck in loop). Cost per conversation alert if $> \$0.08$.

Step 8 | A/B Testing

Experiment design:

- **Unit:** Customer session (not message — session context matters)
- **Treatment vs. control:** New intent classifier / new LLM model vs. current production
- **Duration:** 7 days — customer support is high volume; effects detectable faster than most ML systems
- **Severity bias:** The most complex conversations get escalated to humans regardless. This makes the AI-resolved pool look easier than it is. Track resolution rate separately for each intent type.
- **Decision rule:** Promote if resolution rate improves $\geq$1% and CSAT does not drop, and cost per conversation stays $< \$0.05$.

Step 9 | Deployment and Monitoring

Deployment

- Stateless agent pods behind load balancer
- Session state in Redis (not in pod memory)
- Circuit breaker on all external tool APIs
- Canary rollout for new LLM models (10% traffic first)
- Hard limits enforced in agent runner: max 15 turns, max \$0.10/session

8 Metrics to Monitor

1. Turns-per-conversation distribution
2. Tool API error rates by tool
3. Escalation rate by intent type
4. LLM cost per hour
5. CSAT by conversation type
6. PII leakage alerts
7. Prompt injection attempts
8. Refund approval queue length

Step 10 | Risk Management and Trade-offs

Decision	Choice	Alternative	Why this choice
Autonomous actions	Tiered (auto for low-risk, human for high-risk)	Fully autonomous	Fully autonomous risks unwarranted refunds and policy violations. Tiered balances speed vs. safety.
Single LLM vs. multi-model	Routing to specialised models (classifier + main LLM)	One large LLM for everything	Intent classifier at 20ms is $250\times$ cheaper than using GPT-4o for every routing decision. Saves \$5+/hr at scale.
Context management	Sliding window + compression	Full conversation history	Full history hits context limit after $\sim$20 turns. Compression is lossy but necessary for long sessions.
Loop prevention	Max turns + repeated action detection + cost ceiling	None (rely on LLM to self-terminate)	LLMs do not reliably self-terminate loops. Hard limits are non-negotiable in production.
When NOT to use agents	High-stakes irreversible actions without override, latency $<$ 100ms, rule-based sufficient	Use agents everywhere	Agents add cost and latency. A rule that says "if order status = delivered, reply with tracking URL" does not need an agent.

Failure Modes and Recovery

What breaks	How to detect	Recovery
Infinite loop	turns $>$ 8 in session; same tool called twice	Hard cap at 15 turns; repeated action detector fires; force graceful escalation
Context window overflow	LLM returns context_length_exceeded error	Sliding window compressor activates; summarise oldest 50% of turns
Tool API timeout	Latency spike on tool call; circuit breaker fires	Return cached result if available; inform customer of delay; escalate if tool needed for resolution
Prompt injection attack	Known injection patterns detected in input; safety filter fires	Block response; log event; escalate to human; flag customer account for review
Cost blow-up	Per-session cost tracker $>$ \$0.10	Immediate forced escalation; alert on-call; investigate which intent type caused it